

=====

GCI – GYKHAMINE CONCEPT INVESTIGATION
COURS EXHAUSTIF : IMAGE · VIDÉO · SON AVEC PYTHON
Infographie · Édition Vidéo · Traitement Audio

=====

Auteur : GCI – Département Formation & Intelligence Analytique
Version : 1.0 – Édition Complète
Format : Bible Professionnelle
Langue : Français

=====

TABLE DES MATIÈRES

MODULE A – INFOGRAPHIE ET TRAITEMENT D'IMAGE

PARTIE A-I – FONDAMENTAUX DE L'IMAGE NUMÉRIQUE
PARTIE A-II – PILLOW (PIL) : MANIPULATION SIMPLE ET CRÉATIVE
PARTIE A-III – OPENCV (CV2) : TRAITEMENT AVANCÉ ET VISION PAR ORDINATEUR
PARTIE A-IV – DESSINER ET ANNOTER : TEXTES, FORMES, OVERLAYS
PARTIE A-V – TRAITEMENTS AVANCÉS : FILTRES, DÉTECTION, SEGMENTATION
PARTIE A-VI – INFOGRAPHIE AUTOMATISÉE : TEMPLATES, RAPPORTS VISUELS
PARTIE A-VII – PROJETS MÉTIER ET PIPELINES IMAGE

MODULE B – ÉDITION VIDÉO AVEC PYTHON

PARTIE B-I – FONDAMENTAUX DE LA VIDÉO NUMÉRIQUE
PARTIE B-II – OPENCV (CV2) : LECTURE, TRAITEMENT FRAME PAR FRAME
PARTIE B-III – MOVIEPY : MONTAGE, EFFETS, TRANSITIONS, EXPORT
PARTIE B-IV – FFMPEG LINUX : OUTIL UNIVERSEL EN LIGNE DE COMMANDE
PARTIE B-V – COMBINAISONS AVANCÉES : TEXTES, SOUS-TITRES, OVERLAYS
PARTIE B-VI – PROJETS MÉTIER : RAPPORTS VIDÉO, AUTOMATISATION

MODULE C – ÉDITION ET TRAITEMENT DU SON

PARTIE C-I – FONDAMENTAUX DE L'AUDIO NUMÉRIQUE
PARTIE C-II – PYAUDIO : ENREGISTREMENT ET LECTURE EN TEMPS RÉEL
PARTIE C-III – PYDUB : MANIPULATION, DÉCOUPE, MIXAGE, EFFETS
PARTIE C-IV – FFMPEG AUDIO : CONVERSION, EXTRACTION, COMPRESSION
PARTIE C-V – ANALYSE AUDIO : WAVEFORM, SPECTROGRAMME, FRÉQUENCES
PARTIE C-VI – PROJETS MÉTIER : TRANSCRIPTION, PODCAST, AUTOMATISATION

MODULE A – INFOGRAPHIE ET TRAITEMENT D'IMAGE AVEC PYTHON

PARTIE A-I – FONDAMENTAUX DE L'IMAGE NUMÉRIQUE

A-I.1 CE QU'EST UNE IMAGE EN MÉMOIRE

Une image numérique est un tableau tridimensionnel de valeurs entières.
Chaque cellule est un pixel. Chaque pixel contient des canaux de couleur.

STRUCTURE :

image[hauteur][largeur][canal]

- hauteur : nombre de lignes de pixels (axe Y)
- largeur : nombre de colonnes de pixels (axe X)
- canal : composante de couleur (R, G, B, A...)

MODES COURANTS :

- Niveaux de gris (L) : 1 canal, valeurs 0 (noir) à 255 (blanc)
- RGB : 3 canaux, Rouge + Vert + Bleu

- RGBA : 4 canaux, RGB + transparence Alpha
- HSV : Teinte + Saturation + Valeur (meilleur pour filtres)
- BGR : format natif OpenCV – ordre inversé de RGB (piège !)
- CMYK : usage impression professionnelle

EXEMPLE CONCRET :

Image RGB 1920×1080 = tableau NumPy de shape (1080, 1920, 3)

Taille brute = 1080 × 1920 × 3 = 6 220 800 octets ≈ 6 Mo non compressé

FORMATS DE FICHIERS :

Format	Compression	Transparence	Usage typique
JPEG	Avec perte	Non	Photos, web, petite taille
PNG	Sans perte	Oui (Alpha)	Infographies, logos, UI
BMP	Aucune	Non	Compatibilité maximale
TIFF	Sans perte	Oui	Médical, impression
WEBP	Avec/sans	Oui	Web moderne, compact
GIF	Avec perte	Oui (1 bit)	Animations simples

A-I.2 INSTALLATION ET IMPORTS

INSTALLATION :

```

pip install opencv-python          # OpenCV sans GUI
pip install opencv-contrib-python  # OpenCV + modules contrib (SIFT, etc.)
pip install Pillow                 # PIL Fork (Pillow)
pip install numpy matplotlib      # Tableaux et visualisation

```

IMPORTS STANDARDS :

```

import cv2
import numpy as np
from PIL import Image, ImageDraw, ImageFont, ImageEnhance
from PIL import ImageFilter, ImageOps
import matplotlib.pyplot as plt

```

DIFFÉRENCES FONDAMENTALES :

Critère	Pillow (PIL)	OpenCV (cv2)
Ordre des canaux	RGB	BGR (inversé !)
Type de données	Objet PIL.Image	Tableau NumPy
Interface	Haute abstraction	Bas niveau, très puissant
Dessin / Texte	Simple, élégant	Précis, performant
Filtres / Vision	Basique	Complet (détection, etc.)
Export/Import	Large support format	JPEG/PNG principalement
Usage recommandé	Retouche, création	Analyse, vision, détection

CONVERSION ENTRE LES DEUX (indispensable) :

```

# PIL → OpenCV
pil_img = Image.open("photo.jpg").convert("RGB")
cv_img = np.array(pil_img)
cv_img = cv2.cvtColor(cv_img, cv2.COLOR_RGB2BGR)

# OpenCV → PIL
cv_img_rgb = cv2.cvtColor(cv_img, cv2.COLOR_BGR2RGB)
pil_img = Image.fromarray(cv_img_rgb)

```

RÈGLE D'OR :

Pillow pour créer, retoucher, exporter.

OpenCV pour analyser, détecter, traiter en masse.

Les deux se combinent : traitez avec OpenCV, exportez avec Pillow.

PARTIE A-II – PILLOW (PIL) : MANIPULATION SIMPLE ET CRÉATIVE

A-II.1 OUVRIR, INSPECTER, SAUVEGARDER

```

from PIL import Image

# Ouvrir
img = Image.open("photo.jpg")

# Inspecter
print(img.size)      # (largeur, hauteur) → (1920, 1080)
print(img.mode)      # 'RGB', 'RGBA', 'L', etc.
print(img.format)    # 'JPEG', 'PNG', None si créé en mémoire

# Convertir le mode
img_gray = img.convert("L")      # Niveaux de gris
img_rgba = img.convert("RGBA")   # Ajouter canal Alpha

# Sauvegarder
img.save("sortie.png")
img.save("sortie.jpg", quality=90, optimize=True)
img.save("sortie.webp", quality=85)

# Créer une image vierge
blanc = Image.new("RGB", (800, 600), color=(255, 255, 255))
noir = Image.new("RGB", (800, 600), color=(0, 0, 0))
transp = Image.new("RGBA", (800, 600), color=(0, 0, 0, 0))

```

A-II.2 REDIMENSIONNER ET RECADRER

```

from PIL import Image

img = Image.open("photo.jpg")

# Redimensionner (peut déformer si ratio différent)
img_r = img.resize((800, 600), Image.LANCZOS)

# Redimensionner en conservant le ratio (thumbnail modifie en place)
img_t = img.copy()
img_t.thumbnail((800, 800), Image.LANCZOS)

# Recadrer – box = (gauche, haut, droite, bas)
zone = img.crop((100, 50, 500, 400))

# Rotation
img_rot = img.rotate(45, expand=True) # expand=True ajuste le canvas

# Retournement
from PIL import ImageOps
img_miroir = ImageOps.mirror(img) # Horizontal
img_flip = ImageOps.flip(img)    # Vertical

# Recadrage centré (utile pour thumbnails uniformes)
def crop_center(img, target_w, target_h):
    w, h = img.size
    left = (w - target_w) // 2
    top = (h - target_h) // 2
    right = left + target_w
    bottom = top + target_h
    return img.crop((left, top, right, bottom))

img_carre = crop_center(img, 500, 500)

FILTRES DE RÉÉCHANTILLONNAGE :
– Image.NEAREST : rapide, pixelisé (miniatures, pixel art)
– Image.BILINEAR : interpolation bilinéaire, rapide et correct
– Image.BICUBIC : bonne qualité, plus lent
– Image.LANCZOS : qualité maximale, recommandé pour agrandissements

```

A-II.3 COULEURS, AJUSTEMENTS ET FILTRES

```

from PIL import Image, ImageEnhance, ImageFilter, ImageOps

img = Image.open("photo.jpg").convert("RGB")

```

```

# — Ajustements avec ImageEnhance —————
# Valeur 1.0 = original, >1 = augmente, <1 = réduit

img_bright   = ImageEnhance.Brightness(img).enhance(1.5)
img_contrast = ImageEnhance.Contrast(img).enhance(1.3)
img_color    = ImageEnhance.Color(img).enhance(2.0)      # Saturation
img_sharp    = ImageEnhance.Sharpness(img).enhance(2.0)

# — Transformations colorimétriques —————
img_equalized = ImageOps.equalize(img)      # Égalisation d'histogramme
img_inverted  = ImageOps.invert(img)        # Négatif
img_posterized = ImageOps.posterize(img, 3)  # Réduction de couleurs
img_solarized = ImageOps.solarize(img, 128)  # Effet solarisation

# — Filtres prédéfinis —————
img_blur      = img.filter(ImageFilter.BLUR)
img_sharp2    = img.filter(ImageFilter.SHARPEN)
img_edge      = img.filter(ImageFilter.FIND_EDGES)
img_contour   = img.filter(ImageFilter.CONTOUR)
img_emboss    = img.filter(ImageFilter.EMBOSS)
img_smooth    = img.filter(ImageFilter.SMOOTH_MORE)

# Flou gaussien avec rayon personnalisé
img_gauss = img.filter(ImageFilter.GaussianBlur(radius=5))

# Flou médian (réduction de bruit, préserve les contours)
img_median = img.filter(ImageFilter.MedianFilter(size=5))

# — Manipulation directe des pixels —————
import numpy as np

arr = np.array(img)          # shape (H, W, 3)
arr[:, :, 0] = 0             # Supprimer canal Rouge
img_sans_rouge = Image.fromarray(arr)

# Appliquer une teinte bleue
arr = np.array(img).astype(np.float32)
arr[:, :, 2] = np.clip(arr[:, :, 2] * 1.5, 0, 255)
img_bleu = Image.fromarray(arr.astype(np.uint8))

```

A-II.4 DESSINER SUR UNE IMAGE AVEC IMAGEDRAW

```

from PIL import Image, ImageDraw, ImageFont

img = Image.new("RGB", (800, 400), color=(30, 30, 30))
draw = ImageDraw.Draw(img)

# — Formes géométriques —————

# Rectangle – (x0, y0, x1, y1)
draw.rectangle([50, 50, 300, 200], fill=(70, 130, 180), outline=(255,255,255), width=3)

# Cercle / Ellipse
draw.ellipse([350, 50, 550, 250], fill=(220, 50, 50), outline=(255,255,255), width=2)

# Ligne
draw.line([0, 0, 800, 400], fill=(255, 215, 0), width=2)

# Polygone
draw.polygon([(600, 50), (750, 50), (750, 200), (600, 200)], fill=(50, 200, 50))

# Point
draw.point((400, 200), fill=(255, 255, 255))

# Arc et secteur
draw.arc([650, 250, 780, 380], start=0, end=270, fill=(255,165,0), width=3)
draw.pieslice([650, 250, 780, 380], start=270, end=360, fill=(255, 165, 0))

# — Texte —————

# Texte simple (police par défaut)
draw.text((50, 300), "Bonjour GCI", fill=(255, 255, 255))

```

```
# Texte avec police TrueType
try:
    font_titre = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 36)
    font_normal = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 20)
except IOError:
    font_titre = ImageFont.load_default()
    font_normal = ImageFont.load_default()

draw.text((50, 320), "ANALYSE GCI 2025", fill=(184, 145, 42), font=font_titre)
draw.text((50, 370), "Données Congo-Brazzaville", fill=(200, 200, 200), font=font_normal)

# Mesurer le texte (pour centrage)
bbox = draw.textbbox((0, 0), "Titre centré", font=font_titre)
text_w = bbox[2] - bbox[0]
text_h = bbox[3] - bbox[1]
x_center = (800 - text_w) // 2
draw.text((x_center, 10), "Titre centré", fill=(255, 255, 255), font=font_titre)

img.save("infographie.png")
```

A-II.5 COMPOSITION D'IMAGES : COLLER, FUSIONNER, TRANSPARENCE

```
from PIL import Image

fond = Image.open("fond.jpg").convert("RGBA")
logo = Image.open("logo.png").convert("RGBA")

# Coller une image sur une autre (avec masque alpha)
position = (50, 50)
fond.paste(logo, position, mask=logo) # mask=logo utilise le canal Alpha

# Superposition avec transparence contrôlée
calque = Image.new("RGBA", fond.size, (0, 0, 0, 0))
calque.paste(logo, position)
resultat = Image.alpha_composite(fond, calque)

# Sauvegarder (RGBA nécessite PNG)
resultat.save("composition.png")

# Fusionner deux images en fondu (blend)
img1 = Image.open("image1.jpg").convert("RGB").resize((800, 600))
img2 = Image.open("image2.jpg").convert("RGB").resize((800, 600))
fondu = Image.blend(img1, img2, alpha=0.5) # 0 = img1, 1 = img2
fondu.save("fondu.jpg")

# Filigrane (watermark) semi-transparent
def ajouter_filigrane(img_path, texte, sortie_path):
    img = Image.open(img_path).convert("RGBA")
    calque = Image.new("RGBA", img.size, (255, 255, 255, 0))
    draw = ImageDraw.Draw(calque)
    try:
        font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 40)
    except:
        font = ImageFont.load_default()
    w, h = img.size
    draw.text((w//2 - 100, h//2), texte, fill=(255,255,255,100), font=font)
    fusionné = Image.alpha_composite(img, calque)
    fusionné.convert("RGB").save(sortie_path)

ajouter_filigrane("photo.jpg", "© GCI 2025", "photo_watermark.jpg")
```

PARTIE A-III – OPENCV (CV2) : TRAITEMENT AVANCÉ

A-III.1 OUVRIR, AFFICHER, SAUVEGARDER AVEC CV2

```
import cv2
import numpy as np

# Lire une image (toujours en BGR par défaut)
```

```

img = cv2.imread("photo.jpg")
img_gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
img_alpha = cv2.imread("logo.png", cv2.IMREAD_UNCHANGED) # RGBA

# Dimensions
h, w, c = img.shape # hauteur, largeur, canaux
print(f"{w}x{h}, {c} canaux")

# Afficher (environnement bureau uniquement)
cv2.imshow("Image", img)
cv2.waitKey(0) # Attendre une touche
cv2.destroyAllWindows()

# Sauvegarder
cv2.imwrite("sortie.jpg", img)
cv2.imwrite("sortie.png", img)
cv2.imwrite("sortie.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 90])

# Créer une image vierge
noir = np.zeros((600, 800, 3), dtype=np.uint8)
blanc = np.ones((600, 800, 3), dtype=np.uint8) * 255

# Accéder et modifier des pixels
pixel = img[100, 200] # Pixel à ligne 100, colonne 200 → [B, G, R]
img[100, 200] = [255, 0, 0] # Définir en bleu (BGR)

# Accéder à une région (ROI = Region Of Interest)
roi = img[50:200, 100:400] # Découpe de lignes 50–200, colonnes 100–400

```

A-III.2 CONVERSIONS COLORIMÉTRIQUES

```

import cv2

img = cv2.imread("photo.jpg")

# BGR → Niveaux de gris
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# BGR → RGB (pour matplotlib ou PIL)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# BGR → HSV (détection de couleurs)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# BGR → LAB (perception humaine, meilleur pour comparaisons)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

# BGR → YCrCb (traitement peau humaine)
ycr = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)

POURQUOI HSV ?
En RGB, isoler une couleur est complexe car R, G, B varient ensemble
avec la lumière. En HSV :
– H (Hue) : la teinte pure, indépendante de la luminosité
– S (Saturation) : l'intensité de la couleur
– V (Value) : la luminosité

Exemple – Isoler la couleur rouge :
lower_red = np.array([0, 120, 70])
upper_red = np.array([10, 255, 255])
masque = cv2.inRange(hsv, lower_red, upper_red)
resultat = cv2.bitwise_and(img, img, mask=masque)

```

A-III.3 FILTRES ET TRANSFORMATIONS SPATIALES

```

import cv2
import numpy as np

img = cv2.imread("photo.jpg")

# — Flous

```

```

# Flou moyen (simple, rapide)
blur_moyen = cv2.blur(img, (5, 5))

# Flou gaussien (naturel, réduction de bruit)
blur_gauss = cv2.GaussianBlur(img, (15, 15), 0)

# Flou médian (supprime le bruit sel & poivre)
blur_median = cv2.medianBlur(img, 5)

# Flou bilatéral (préserve les contours – excellent pour portraits)
blur_bilat = cv2.bilateralFilter(img, 9, 75, 75)

# — Détection de contours —————

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Canny – paramètres : seuil bas, seuil haut
contours_canny = cv2.Canny(gray, 100, 200)

# Sobel – gradient horizontal et vertical
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=5)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=5)
sobel   = cv2.magnitude(sobel_x, sobel_y)

# Laplacian
laplacian = cv2.Laplacian(gray, cv2.CV_64F)

# — Transformations géométriques —————

h, w = img.shape[:2]

# Redimensionner
img_r = cv2.resize(img, (640, 480))
img_r = cv2.resize(img, None, fx=0.5, fy=0.5) # 50% de la taille

# Rotation
centre = (w // 2, h // 2)
matrice = cv2.getRotationMatrix2D(centre, angle=45, scale=1.0)
img_rot = cv2.warpAffine(img, matrice, (w, h))

# Retournement
img_flip_h = cv2.flip(img, 1) # Horizontal
img_flip_v = cv2.flip(img, 0) # Vertical
img_flip_b = cv2.flip(img, -1) # Les deux

# Recadrage
crop = img[100:400, 150:500] # [lignes, colonnes]

# Transformation affine (3 points correspondants)
pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])
M_aff = cv2.getAffineTransform(pts1, pts2)
img_aff = cv2.warpAffine(img, M_aff, (w, h))

```

A-III.4 DESSIN AVEC OPENCV

```

import cv2
import numpy as np

canvas = np.zeros((600, 800, 3), dtype=np.uint8)

# Ligne – (image, pt1, pt2, couleur_BGR, épaisseur)
cv2.line(canvas, (0, 0), (800, 600), (0, 255, 0), 2)

# Rectangle – (image, coin_haut_gauche, coin_bas_droit, couleur, épaisseur)
# épaisseur = -1 pour remplir
cv2.rectangle(canvas, (50, 50), (300, 200), (255, 0, 0), 3)
cv2.rectangle(canvas, (350, 50), (600, 200), (0, 0, 255), -1)

# Cercle – (image, centre, rayon, couleur, épaisseur)
cv2.circle(canvas, (400, 400), 80, (0, 255, 255), -1)

```

```
# Ellipse
cv2.ellipse(canvas, (650, 400), (100, 50), 45, 0, 360, (255, 165, 0), -1)

# Polygone
points = np.array([[100,400],[150,300],[200,400],[250,300],[300,400]])
cv2.polylines(canvas, [points], isClosed=False, color=(255,255,255), thickness=2)

# Texte
cv2.putText(
    canvas,
    "GCI Analytics",
    (50, 550),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.5,
    (184, 145, 42),
    2,
    cv2.LINE_AA
)
# Position
# Police
# Échelle
# Couleur BGR
# Épaisseur
# Anti-aliasing

POLICES DISPONIBLES DANS CV2 :
- cv2.FONT_HERSHEY_SIMPLEX      : standard
- cv2.FONT_HERSHEY_PLAIN       : petite taille
- cv2.FONT_HERSHEY_DUPLEX      : double trait
- cv2.FONT_HERSHEY_COMPLEX     : plus élaboré
- cv2.FONT_HERSHEY_TRIPLEX     : triple trait, épais
- cv2.FONT_HERSHEY_SCRIPT_SIMPLEX : écriture cursive simple
- cv2.FONT_ITALIC              : italique (combinable)
```

A-III.5 DÉTECTION DE VISAGES ET D'OBJETS (HAAR CASCADE)

```
import cv2

# Charger le classifieur pré-entraîné (inclus dans opencv)
cascade_visage = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

img = cv2.imread("photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Détecter les visages
# scaleFactor : réduction de l'image à chaque passage (1.1 = 10%)
# minNeighbors : nombre de détections voisines minimales (évite faux positifs)
visages = cascade_visage.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30, 30)
)

# Annoter chaque visage détecté
for (x, y, w, h) in visages:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.putText(img, "Visage", (x, y-10),
        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,0), 2)

print(f"{len(visages)} visage(s) détecté(s)")
cv2.imwrite("visages_detectes.jpg", img)

AUTRES CLASSIFIEURS DISPONIBLES :
- haarcascade_eye.xml
- haarcascade_smile.xml
- haarcascade_fullbody.xml
- haarcascade_upperbody.xml
- haarcascade_profileface.xml
```

A-III.6 SEUILLAGE ET SEGMENTATION

```
import cv2
import numpy as np
```



```

img = cv2.imread("photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# — Seuillage simple —————
# Pixels > seuil → 255 (blanc), autres → 0 (noir)
_, thresh_bin = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
_, thresh_inv = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY_INV)
_, thresh_trunc = cv2.threshold(gray, 127, 255, cv2.THRESH_TRUNC)
_, thresh_otsu = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Seuillage adaptatif (pour éclairages inégaux)
thresh_adapt = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    blockSize=11,
    C=2
)

# — Opérations morphologiques —————
kernel = np.ones((5, 5), np.uint8)

erosion = cv2.erode(thresh_bin, kernel, iterations=1) # Réduit les zones blanches
dilatation = cv2.dilate(thresh_bin, kernel, iterations=1) # Agrandit les zones blanches
ouverture = cv2.morphologyEx(thresh_bin, cv2.MORPH_OPEN, kernel) # Erosion + Dilatation
fermeture = cv2.morphologyEx(thresh_bin, cv2.MORPH_CLOSE, kernel) # Dilatation + Erosion
gradient = cv2.morphologyEx(thresh_bin, cv2.MORPH_GRADIENT, kernel)

# — Détection de contours (findContours) —————
contours, hierarchy = cv2.findContours(
    thresh_bin,
    cv2.RETR_EXTERNAL, # Contours externes seulement
    cv2.CHAIN_APPROX_SIMPLE # Compression des points redondants
)

# Filtrer par surface
grands_contours = [c for c in contours if cv2.contourArea(c) > 500]

# Dessiner
img_copy = img.copy()
cv2.drawContours(img_copy, grands_contours, -1, (0, 255, 0), 2)

# Boîtes englobantes
for c in grands_contours:
    x, y, w, h = cv2.boundingRect(c)
    cv2.rectangle(img_copy, (x, y), (x+w, y+h), (255, 0, 0), 2)

```

PARTIE A-IV – INFOGRAPHIE AUTOMATISÉE ET PIPELINES MÉTIER

A-IV.1 GÉNÉRATION AUTOMATIQUE DE VISUELS (RAPPORTS, CARTES, AFFICHES)

PRINCIPE :

L'infographie automatisée consiste à générer des images à partir de données, sans intervention manuelle. Un script Python produit un visuel prêt à l'emploi.

CAS D'USAGE MÉTIER :

- Rapport quotidien automatique avec indicateurs clés
- Cartes de performance pour chaque commercial
- Affiches de résultats pour un tableau de bord physique
- Génération de certificats personnalisés en masse
- Vignettes sociales (social cards) générées depuis une base de données

EXEMPLE COMPLET – CARTE DE KPI :

```

from PIL import Image, ImageDraw, ImageFont
import numpy as np

def generer_carte_kpi(titre, valeur, sous_titre, couleur_fond, chemin_sortie):
    W, H = 600, 300
    img = Image.new("RGB", (W, H), color=couleur_fond)
    draw = ImageDraw.Draw(img)

```

```

try:
    font_titre = ImageFont.truetype("DejaVuSans-Bold.ttf", 28)
    font_valeur = ImageFont.truetype("DejaVuSans-Bold.ttf", 72)
    font_sous = ImageFont.truetype("DejaVuSans.ttf", 18)
except:
    font_titre = font_valeur = font_sous = ImageFont.load_default()

# Bande latérale décorative
draw.rectangle([0, 0, 12, H], fill=(184, 145, 42))

# Titre
draw.text((30, 30), titre, fill=(220, 220, 220), font=font_titre)

# Valeur principale (centrée)
bbox = draw.textbbox((0,0), valeur, font=font_valeur)
tw = bbox[2] - bbox[0]
draw.text(((W - tw) // 2, 100), valeur, fill=(255, 255, 255), font=font_valeur)

# Sous-titre
draw.text((30, H - 50), sous_titre, fill=(180, 180, 180), font=font_sous)

# Ligne de séparation bas
draw.line([30, H-65, W-30, H-65], fill=(184, 145, 42), width=1)

img.save(chemin_sortie)

generer_carte_kpi(
    titre="Chiffre d'Affaires",
    valeur="4 820 000 CFA",
    sous_titre="Mois d'Avril 2025 – Brazzaville",
    couleur_fond=(13, 27, 42),
    chemin_sortie="kpi_ca.png"
)

```

A-IV.2 TRAITEMENT EN MASSE D'IMAGES (BATCH PROCESSING)

```

import os
from PIL import Image

dossier_entree = "images_brutes/"
dossier_sortie = "images_traitees/"
os.makedirs(dossier_sortie, exist_ok=True)

extensions_valides = (".jpg", ".jpeg", ".png", ".bmp", ".webp")

for fichier in os.listdir(dossier_entree):
    if not fichier.lower().endswith(extensions_valides):
        continue

    chemin = os.path.join(dossier_entree, fichier)

    try:
        img = Image.open(chemin).convert("RGB")

        # Redimensionner en 800px max
        img.thumbnail((800, 800), Image.LANCZOS)

        # Nom de sortie
        nom, _ = os.path.splitext(fichier)
        chemin_sortie = os.path.join(dossier_sortie, nom + ".jpg")

        img.save(chemin_sortie, "JPEG", quality=85, optimize=True)
        print(f"✓ {fichier} → {chemin_sortie}")

    except Exception as e:
        print(f"✗ Erreur {fichier} : {e}")

```

A-IV.3 ERREURS FRÉQUENTES ET BONNES PRATIQUES

ERREUR 1 – Confusion RGB / BGR

- OpenCV lit en BGR, Pillow en RGB. Toujours convertir avant d'échanger.
- `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` avant de passer à PIL ou matplotlib.

ERREUR 2 – Afficher avec matplotlib une image OpenCV

- `plt.imshow(img)` affiche les canaux dans l'ordre reçu → couleurs fausses.
- Toujours faire : `plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))`

ERREUR 3 – Sauvegarder RGBA en JPEG

- JPEG ne supporte pas la transparence → erreur ou canal perdu.
- `img.convert("RGB").save("sortie.jpg")`

ERREUR 4 – Modifier l'image originale sans copie

- `img_modif = img.copy()` avant toute modification destructive.

ERREUR 5 – Type de données NumPy incorrect

- Les opérations mathématiques peuvent produire des float64 hors de [0,255].
- Toujours faire `arr.astype(np.uint8)` avec `np.clip(arr, 0, 255)` avant.

BONNES PRATIQUES :

- Travailler toujours sur des copies des images originales
- Vérifier `img is not None` après `cv2.imread()` (chemin invalide → None)
- Utiliser `os.path.exists()` avant d'ouvrir un fichier
- Logguer les erreurs en batch plutôt que d'arrêter le pipeline
- Choisir PNG pour les images avec texte/graphiques, JPEG pour les photos

MODULE B – ÉDITION VIDÉO AVEC PYTHON

PARTIE B-I – FONDAMENTAUX DE LA VIDÉO NUMÉRIQUE

B-I.1 STRUCTURE D'UN FICHIER VIDÉO

Une vidéo numérique est une séquence d'images (frames) associée à une ou plusieurs pistes audio, emballées dans un conteneur.

COMPOSANTS FONDAMENTAUX :

CONTENEUR (Container)

- Fichier qui encapsule video, audio, sous-titres, métadonnées
- Exemples : MP4, AVI, MKV, MOV, WebM

CODEC VIDÉO

- Algorithme de compression/décompression des frames
- Exemples : H.264 (AVC), H.265 (HEVC), VP9, AV1, MPEG-4

CODEC AUDIO

- Algorithme de compression du son
- Exemples : AAC, MP3, Opus, FLAC, PCM

FLUX (Streams)

- Une vidéo MP4 contient généralement : 1 flux vidéo + 1 flux audio
- Un MKV peut contenir plusieurs pistes audio et sous-titres

PARAMÈTRES CLÉS :

Paramètre	Définition	Exemple
Résolution	Largeur × Hauteur en pixels	1920×1080
FPS	Frames Per Second (images/seconde)	24, 30, 60
Bitrate vidéo	Données par seconde (kbps ou Mbps)	5 Mbps
Durée	Durée totale	01:32:10
Codec vidéo	Algorithme de compression	H.264
Codec audio	Algorithme audio	AAC 128kbps
SAR / DAR	Ratio d'aspect pixels / affichage	16:9

QUALITÉ DES CODECS (du moins au plus efficace en compression) :

MPEG-2 < MPEG-4 < H.264 < H.265 < AV1

INSTALLATION DES OUTILS :

```
pip install opencv-python      # Lecture/écriture vidéo frame par frame
pip install moviepy            # Montage, effets, transitions
sudo apt install ffmpeg       # FFmpeg (outil système Linux)
```

B-I.2 FFMPEG : VÉRIFIER L'INSTALLATION

LINUX :

```
sudo apt update && sudo apt install ffmpeg -y
ffmpeg -version
```

VÉRIFICATION DEPUIS PYTHON :

```
import subprocess
result = subprocess.run(["ffmpeg", "-version"], capture_output=True, text=True)
print(result.stdout[:200])
```

INFORMATIONS SUR UN FICHER VIDÉO :

```
ffprobe -v quiet -print_format json -show_streams video.mp4
```

PARTIE B-II – OPENCV (CV2) : LECTURE ET TRAITEMENT FRAME PAR FRAME

B-II.1 LIRE ET PARCOURIR UNE VIDÉO

```
import cv2

cap = cv2.VideoCapture("video.mp4")

if not cap.isOpened():
    print("Erreur : impossible d'ouvrir la vidéo")
    exit()

# Propriétés de la vidéo
fps = cap.get(cv2.CAP_PROP_FPS)
largeur = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
hauteur = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
duree = total / fps

print(f"Résolution : {largeur}x{hauteur}")
print(f"FPS : {fps}")
print(f"Frames totales : {total}")
print(f"Durée : {duree:.2f} secondes")

# Lire frame par frame
while True:
    ret, frame = cap.read()

    if not ret:
        break # Fin de la vidéo ou erreur

    # Traiter la frame ici
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Afficher (optionnel, environnement bureau)
    cv2.imshow("Frame", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

B-II.2 ÉCRIRE UNE VIDÉO (EXPORT)

```
import cv2
```

```

cap = cv2.VideoCapture("video_entree.mp4")
fps    = cap.get(cv2.CAP_PROP_FPS)
largeur = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
hauteur = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

# Définir le codec et créer l'objet d'écriture
fourcc = cv2.VideoWriter_fourcc(*"mp4v") # Codec pour .mp4
out     = cv2.VideoWriter("video_sortie.mp4", fourcc, fps, (largeur, hauteur))

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Appliquer un traitement sur chaque frame
    frame_traitee = cv2.GaussianBlur(frame, (15, 15), 0)

    out.write(frame_traitee)

cap.release()
out.release()

CODECS COURANTS :
"mp4v" -> fichier .mp4   (MPEG-4)
"XVID" -> fichier .avi   (DivX/Xvid)
"MJPG" -> fichier .avi   (Motion JPEG)
"avc1" -> fichier .mp4   (H.264, si disponible)

ATTENTION : cv2.VideoWriter n'écrit pas l'audio.
Pour conserver l'audio, utiliser FFmpeg après coup :
ffmpeg -i video_sortie_sans_audio.mp4 -i video_entree.mp4
        -c copy -map 0:v -map 1:a video_finale.mp4

```

B-II.3 EXTRAIRE DES FRAMES ET CRÉER UNE VIDÉO DEPUIS DES IMAGES

```

import cv2
import os

# — Extraire toutes les frames d'une vidéo —————
cap = cv2.VideoCapture("video.mp4")
dossier = "frames/"
os.makedirs(dossier, exist_ok=True)

i = 0
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imwrite(f"{dossier}frame_{i:05d}.jpg", frame)
    i += 1

cap.release()
print(f"{i} frames extraites")

# — Extraire une frame toutes les N secondes —————
cap = cv2.VideoCapture("video.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)
N = 5 # 1 frame toutes les 5 secondes
i = 0

while cap.isOpened():
    frame_pos = int(i * fps * N)
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_pos)
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imwrite(f"capture_{i:03d}.jpg", frame)
    i += 1

cap.release()

# — Créer une vidéo depuis une séquence d'images —————

```

```

import glob

images = sorted(glob.glob("frames/*.jpg"))
if not images:
    exit("Aucune image trouvée")

frame0 = cv2.imread(images[0])
h, w = frame0.shape[:2]

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("timelapse.mp4", fourcc, 24, (w, h))

for chemin in images:
    frame = cv2.imread(chemin)
    out.write(frame)

out.release()
print(f"Vidéo créée depuis {len(images)} images")

```

PARTIE B-III – MOVIEPY : MONTAGE, EFFETS, TRANSITIONS

B-III.1 INTRODUCTION À MOVIEPY

MoviePy est la bibliothèque Python de référence pour le montage vidéo. Elle offre une API haut niveau pour couper, assembler, ajouter du texte, des effets et exporter des vidéos complètes avec audio.

```

pip install moviepy

from moviepy.editor import (
    VideoFileClip, AudioFileClip, ImageClip,
    TextClip, CompositeVideoClip, concatenate_videoclips,
    ColorClip
)

```

B-III.2 OPÉRATIONS DE BASE

```

from moviepy.editor import VideoFileClip

clip = VideoFileClip("video.mp4")

# Propriétés
print(clip.duration) # Durée en secondes
print(clip.fps)      # Images par seconde
print(clip.size)     # (largeur, hauteur)

# — Découpe temporelle —
extrait = clip.subclip(10, 30) # De 10s à 30s
extrait = clip.subclip("00:01:30", "00:02:00") # Format HH:MM:SS

# — Recadrage spatial —
crop = clip.crop(x1=100, y1=50, x2=900, y2=500)
crop = clip.crop(width=640, height=360, x_center=960, y_center=540)

# — Redimensionner —
clip_hd = clip.resize((1920, 1080))
clip_50 = clip.resize(0.5) # 50% de la taille originale

# — Rotation —
clip_rot = clip.rotate(90)

# — Vitesse —
clip_2x = clip.speedx(2) # Accéléré x2
clip_slow = clip.speedx(0.5) # Ralenti x0.5
clip_reverse = clip.fx(vfx.time_mirror) # Inverser la vidéo

# — Export —
extrait.write_videofile(
    "sortie.mp4",

```

```

        codec="libx264",
        audio_codec="aac",
        fps=30
    )

```

B-III.3 ASSEMBLAGE ET TRANSITIONS

```

from moviepy.editor import VideoFileClip, concatenate_videoclips, vfx

clip1 = VideoFileClip("video1.mp4").subclip(0, 10)
clip2 = VideoFileClip("video2.mp4").subclip(5, 20)
clip3 = VideoFileClip("video3.mp4").subclip(0, 8)

# — Concaténation simple —————
final = concatenate_videoclips([clip1, clip2, clip3], method="compose")
final.write_videofile("montage.mp4", codec="libx264", audio_codec="aac")

# — Fondu enchaîné (crossfade) —————
clip1_fade = clip1.crossfadeout(1)    # Fondu sortant sur 1s
clip2_fade = clip2.crossfadein(1)     # Fondu entrant sur 1s

final_fade = concatenate_videoclips(
    [clip1_fade, clip2_fade],
    padding=-1,          # Chevauchement d'1s pour le fondu
    method="compose"
)

# — Fondu au noir (fade to black) —————
clip_fout = clip1.fadeout(1)          # Noir à la fin
clip_fin  = clip2.fadein(1)           # Depuis le noir au début

# — Transition par coupe sèche (par défaut dans concatenate) —————
final = concatenate_videoclips([clip1, clip2, clip3])

```

B-III.4 TEXTE, IMAGES ET SUPERPOSITIONS

```

from moviepy.editor import (
    VideoFileClip, TextClip, ImageClip,
    CompositeVideoClip, ColorClip
)

clip = VideoFileClip("video.mp4")

# — Ajouter du texte —————
# Nécessite ImageMagick : sudo apt install imagemagick

titre = TextClip(
    "RAPPORT GCI 2025",
    fontsize=60,
    color="white",
    font="DejaVu-Sans-Bold",
    stroke_color="black",
    stroke_width=2
)
titre = titre.set_position(("center", 50)).set_duration(5)

# — Sous-titre en bas —————
sous_titre = (TextClip("Congo-Brazzaville", fontsize=30, color="yellow")
    .set_position(("center", "bottom"))
    .set_duration(clip.duration)
    .margin(bottom=30, opacity=0))

# — Superposer un logo (image) —————
logo = (ImageClip("logo.png")
    .set_duration(clip.duration)
    .resize(height=80)
    .set_position(("right", "top"))
    .margin(right=20, top=20, opacity=0))

# — Bande de couleur semi-transparente —————
bande = (ColorClip(size=(clip.w, 80), color=[0, 0, 0])

```

```

        .set_opacity(0.6)
        .set_position(("center", clip.h - 90))
        .set_duration(clip.duration))

# — Composition finale —————
final = CompositeVideoClip([clip, bande, sous_titre, logo, titre])
final.write_videofile("video_annotee.mp4", codec="libx264", audio_codec="aac")

```

B-III.5 GESTION DE L'AUDIO AVEC MOVIEPY

```

from moviepy.editor import VideoFileClip, AudioFileClip

clip = VideoFileClip("video.mp4")

# — Extraire l'audio —————
clip.audio.write_audiofile("audio_extraite.mp3")
clip.audio.write_audiofile("audio_extraite.wav")

# — Remplacer l'audio —————
nouvel_audio = AudioFileClip("musique.mp3").subclip(0, clip.duration)
clip_sans_son = clip.without_audio()
clip_final = clip_sans_son.set_audio(nouvel_audio)
clip_final.write_videofile("video_nouvelle_musique.mp4", codec="libx264", audio_codec="aac")

# — Mixage audio (voix + musique de fond) —————
from moviepy.audio.fx.all import volumex
from moviepy.editor import CompositeAudioClip

voix = AudioFileClip("voix.mp3")
musique = AudioFileClip("fond.mp3").subclip(0, voix.duration).fx(volumex, 0.2)
mixage = CompositeAudioClip([voix, musique])
clip_final = clip.set_audio(mixage)
clip_final.write_videofile("video_mixee.mp4", codec="libx264", audio_codec="aac")

# — Supprimer l'audio —————
clip_muet = clip.without_audio()
clip_muet.write_videofile("video_muette.mp4", codec="libx264")

```

PARTIE B-IV – FFMPEG LINUX : OUTIL UNIVERSEL EN LIGNE DE COMMANDE

B-IV.1 ANATOMIE D'UNE COMMANDE FFMPEG

STRUCTURE GÉNÉRALE :

```
ffmpeg [options_globales] [options_entrée] -i entrée [options_sortie] sortie
```

```

- -i          : fichier d'entrée (Input)
- -c:v        : codec vidéo (copy = sans ré-encodage)
- -c:a        : codec audio
- -vf         : filtre vidéo (video filter)
- -af         : filtre audio (audio filter)
- -ss         : position de début (seek)
- -t          : durée
- -to         : position de fin
- -r          : framerate
- -b:v        : bitrate vidéo
- -b:a        : bitrate audio
- -vn         : pas de vidéo (video none)
- -an         : pas d'audio (audio none)
- -y          : écraser sans confirmation
- -v quiet    : mode silencieux

```

B-IV.2 OPÉRATIONS FONDAMENTALES

```

# Informations sur un fichier
ffprobe -v quiet -print_format json -show_streams video.mp4

```



```
# Convertir le format
ffmpeg -i video.avi -c:v libx264 -c:a aac sortie.mp4

# Découper sans ré-encodage (ultra rapide)
ffmpeg -ss 00:01:00 -to 00:02:30 -i video.mp4 -c copy extrait.mp4

# Découper avec ré-encodage (précision image)
ffmpeg -i video.mp4 -ss 00:01:00 -t 90 -c:v libx264 -c:a aac extrait.mp4

# Changer la résolution
ffmpeg -i video.mp4 -vf scale=1280:720 -c:a copy hd.mp4
ffmpeg -i video.mp4 -vf scale=640:-1 -c:a copy mobile.mp4 # -1 = auto

# Changer le FPS
ffmpeg -i video.mp4 -r 30 -c:a copy video_30fps.mp4

# Changer le bitrate
ffmpeg -i video.mp4 -b:v 2M -b:a 128k réduit.mp4

# Rotation
ffmpeg -i video.mp4 -vf "transpose=1" rotated.mp4
# transpose=0 : rotation 90° anti-horaire + flip vertical
# transpose=1 : rotation 90° horaire
# transpose=2 : rotation 90° anti-horaire
# transpose=3 : rotation 90° horaire + flip vertical

# Retournement
ffmpeg -i video.mp4 -vf hflip sortie.mp4 # Miroir horizontal
ffmpeg -i video.mp4 -vf vflip sortie.mp4 # Miroir vertical
```

B-IV.3 AUDIO AVEC FFMPEG

```
# Extraire l'audio en MP3
ffmpeg -i video.mp4 -vn -c:a libmp3lame -q:a 2 audio.mp3

# Extraire l'audio en WAV (sans perte)
ffmpeg -i video.mp4 -vn -c:a pcm_s16le audio.wav

# Extraire l'audio en AAC
ffmpeg -i video.mp4 -vn -c:a aac -b:a 192k audio.aac

# Remplacer l'audio d'une vidéo
ffmpeg -i video.mp4 -i nouvelle_musique.mp3 -c:v copy -c:a aac -map 0:v -map 1:a sortie.mp4

# Supprimer l'audio
ffmpeg -i video.mp4 -an -c:v copy video_muette.mp4

# Ajouter une piste audio à une vidéo muette
ffmpeg -i video_muette.mp4 -i audio.mp3 -c:v copy -c:a aac -shortest sortie.mp4
```

B-IV.4 ASSEMBLAGE ET MONTAGE AVEC FFMPEG

```
# Concaténer des vidéos de MÊME format/codec (ultra rapide)
# Étape 1 : créer liste.txt
echo "file 'video1.mp4'" > liste.txt
echo "file 'video2.mp4'" >> liste.txt
echo "file 'video3.mp4'" >> liste.txt
# Étape 2 : concaténer
ffmpeg -f concat -safe 0 -i liste.txt -c copy montage.mp4

# Concaténer des vidéos de formats différents (ré-encodage)
ffmpeg -i video1.mp4 -i video2.avi \
  -filter_complex "[0:v][0:a][1:v][1:a]concat=n=2:v=1:a=1[v][a]" \
  -map "[v]" -map "[a]" sortie.mp4

# Créer une vidéo depuis des images
ffmpeg -framerate 24 -i frame_%05d.jpg -c:v libx264 -pix_fmt yuv420p timelapse.mp4

# Créer une vidéo depuis une image + audio (slideshow)
ffmpeg -loop 1 -i image.jpg -i audio.mp3 -c:v libx264 -tune stillimage \
  -c:a aac -b:a 192k -shortest video_image.mp4
```

```
# Ajouter un texte/filigrane
ffmpeg -i video.mp4 -vf \
    "drawtext=text='GCI 2025':fontcolor=white:fontsize=40:x=30:y=30" \
    sortie.mp4

# Texte en bas centré avec fond semi-transparent
ffmpeg -i video.mp4 -vf \
    "drawbox=y=ih-80:color=black@0.5:width=iw:height=80:t=fill,
    drawtext=text='© GCI Congo-Brazzaville':fontcolor=white:fontsize=28:
    x=(w-text_w)/2:y=h-55" \
    sortie.mp4

# Convertir en niveaux de gris
ffmpeg -i video.mp4 -vf format=gray sortie.mp4

# Flou gaussien
ffmpeg -i video.mp4 -vf gblur=sigma=5 floutee.mp4

# Accélérer ou ralentir
ffmpeg -i video.mp4 -vf "setpts=0.5*PTS" -af "atempo=2.0" video_2x.mp4
ffmpeg -i video.mp4 -vf "setpts=2.0*PTS" -af "atempo=0.5" video_lente.mp4

# Recadrer
ffmpeg -i video.mp4 -vf "crop=640:360:100:50" sortie.mp4
# format : crop=w:h:x:y

# Stabilisation vidéo (nécessite ffmpeg-vidstab)
ffmpeg -i video.mp4 -vf vidstabdetect=result=transform.trf -f null -
ffmpeg -i video.mp4 -vf vidstabtransform=input=transform.trf stabilisee.mp4
```

```
import subprocess
import os

def ffmpeg_run(commande, afficher=True):
    """Exécuter une commande FFmpeg depuis Python."""
    if afficher:
        print("→ " + " ".join(commande))
    result = subprocess.run(commande, capture_output=True, text=True)
    if result.returncode != 0:
        print("ERREUR FFmpeg :")
        print(result.stderr[-500:])
    return result.returncode == 0

# Exemples d'utilisation

# Convertir
ffmpeg_run(["ffmpeg", "-y", "-i", "entree.avi", "-c:v", "libx264",
            "-c:a", "aac", "sortie.mp4"])

# Extraire audio
ffmpeg_run(["ffmpeg", "-y", "-i", "video.mp4", "-vn",
            "-c:a", "libmp3lame", "-q:a", "2", "audio.mp3"])

# Redimensionner
ffmpeg_run(["ffmpeg", "-y", "-i", "video.mp4",
            "-vf", "scale=1280:720", "-c:a", "copy", "hd.mp4"])

# Batch : convertir tous les .avi en .mp4 d'un dossier
dossier = "videos_brutes/"
for f in os.listdir(dossier):
    if f.endswith(".avi"):
        entree = os.path.join(dossier, f)
        sortie = os.path.join("videos_mp4", f.replace(".avi", ".mp4"))
        os.makedirs("videos_mp4", exist_ok=True)
        ffmpeg_run(["ffmpeg", "-y", "-i", entree,
                    "-c:v", "libx264", "-c:a", "aac", sortie])
```

B-V.1 SURVEILLANCE ET DÉTECTION DE MOUVEMENT

```
import cv2
import time

cap      = cv2.VideoCapture(0) # Webcam (ou chemin vidéo)
back_sub = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=50)
alertes  = 0
seuil_area = 5000 # Pixels minimum pour déclencher une alerte

fps      = cap.get(cv2.CAP_PROP_FPS) or 30
w        = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h        = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fourcc   = cv2.VideoWriter_fourcc(*"mp4v")
out      = None

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    masque = back_sub.apply(frame)
    _, masque = cv2.threshold(masque, 200, 255, cv2.THRESH_BINARY)
    contours, _ = cv2.findContours(masque, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    mouvement = False
    for c in contours:
        if cv2.contourArea(c) > seuil_area:
            mouvement = True
            x, y, cw, ch = cv2.boundingRect(c)
            cv2.rectangle(frame, (x, y), (x+cw, y+ch), (0, 0, 255), 2)

    if mouvement:
        alertes += 1
        horodatage = time.strftime("%Y-%m-%d %H:%M:%S")
        cv2.putText(frame, f"MOUVEMENT DETECTE {horodatage}",
                    (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
        if out is None:
            nom_fichier = f"alerte_{time.strftime('%Y%m%d_%H%M%S')}.mp4"
            out = cv2.VideoWriter(nom_fichier, fourcc, fps, (w, h))
        out.write(frame)
    else:
        if out is not None:
            out.release()
            out = None

cap.release()
if out:
    out.release()
print(f"Total alertes : {alertes}")
```

B-V.2 GÉNÉRATION DE RAPPORT VIDÉO AUTOMATIQUE

CONCEPT :

Générer automatiquement une vidéo de présentation à partir de données.
Chaque slide = une image PIL. L'assemblage = MoviePy ou FFmpeg.

```
from PIL import Image, ImageDraw, ImageFont
import os, subprocess

donnees = [
    {"titre": "Ventes Totales",          "valeur": "12 450 000 CFA", "variation": "+18%"},
    {"titre": "Nouveaux Clients",        "valeur": "342",           "variation": "+7%"},
    {"titre": "Chiffre d'Affaires",      "valeur": "8 900 000 CFA", "variation": "+22%"},
]
```

```

dossier_slides = "slides/"
os.makedirs(dossier_slides, exist_ok=True)

for i, d in enumerate(donnees):
    img = Image.new("RGB", (1280, 720), color=(13, 27, 42))
    draw = ImageDraw.Draw(img)

    try:
        f_big = ImageFont.truetype("DejaVuSans-Bold.ttf", 64)
        f_med = ImageFont.truetype("DejaVuSans-Bold.ttf", 40)
        f_sml = ImageFont.truetype("DejaVuSans.ttf", 28)
    except:
        f_big = f_med = f_sml = ImageFont.load_default()

    # Bande dorée latérale
    draw.rectangle([0, 0, 15, 720], fill=(184, 145, 42))

    # Titre
    draw.text((60, 80), d["titre"], fill=(200, 200, 200), font=f_med)

    # Valeur principale centrée
    bbox = draw.textbbox((0, 0), d["valeur"], font=f_big)
    tw = bbox[2] - bbox[0]
    draw.text(((1280 - tw) // 2, 280), d["valeur"], fill=(255,255,255), font=f_big)

    # Variation
    couleur = (50, 205, 50) if "+" in d["variation"] else (220, 50, 50)
    draw.text((60, 560), f"Variation : {d['variation']}", fill=couleur, font=f_sml)

    # Footer GCI
    draw.line([60, 650, 1220, 650], fill=(184, 145, 42), width=1)
    draw.text((60, 665), "GCI – Gykhamine Concept Investigation – Congo-Brazzaville",
              fill=(150, 150, 150), font=f_sml)

    img.save(f"{dossier_slides}slide_{i:03d}.png")

# Assembler en vidéo (3 secondes par slide)
# Dupliquer chaque image 72 fois (24fps x 3s = 72 frames)
frames_dir = "frames_rapport/"
os.makedirs(frames_dir, exist_ok=True)
frame_idx = 0
for slide_path in sorted(os.listdir(dossier_slides)):
    img = Image.open(os.path.join(dossier_slides, slide_path))
    for _ in range(72):
        img.save(f"{frames_dir}frame_{frame_idx:05d}.png")
        frame_idx += 1

subprocess.run([
    "ffmpeg", "-y", "-framerate", "24",
    "-i", f"{frames_dir}frame_%05d.png",
    "-c:v", "libx264", "-pix_fmt", "yuv420p",
    "rapport_video.mp4"
])
print("Rapport vidéo généré : rapport_video.mp4")

```

MODULE C – ÉDITION ET TRAITEMENT DU SON AVEC PYTHON

PARTIE C-I – FONDAMENTAUX DE L'AUDIO NUMÉRIQUE

C-I.1 STRUCTURE D'UN SIGNAL AUDIO

Le son est une variation de pression de l'air dans le temps.
En numérique, il est représenté par une séquence d'échantillons (samples).

PARAMÈTRES FONDAMENTAUX :

Paramètre	Définition	Valeurs courantes
-----------	------------	-------------------

Fréquence (Hz)	Nombre de cycles par seconde	20 Hz – 20 000 Hz
Sample Rate (Hz)	Échantillons par seconde	44100, 48000 Hz
Bit Depth	Précision d'un sample	16, 24, 32 bits
Canaux	Mono (1) ou Stéréo (2)	1 ou 2
Bitrate (kbps)	Débit de données audio	128, 192, 320 kbps
Durée	Temps total	secondes

THÉORÈME DE NYQUIST :

Pour capter fidèlement une fréquence F , il faut un sample rate $\geq 2F$.

L'oreille humaine perçoit jusqu'à ~20 000 Hz.

→ CD Audio à 44 100 Hz = $2 \times 20\,000$ + marge = fidélité parfaite.

FORMATS AUDIO :

Format	Compression	Qualité	Usage typique
WAV	Aucune (PCM)	Maximale	Studio, traitement
FLAC	Sans perte	Maximale	Archivage, audiophiles
MP3	Avec perte	Bonne	Musique, podcast, web
AAC	Avec perte	Très bonne	Streaming, vidéo
OGG	Avec perte	Bonne	Jeux, web open source
OPUS	Avec perte	Excellente	VoIP, streaming moderne

RÈGLE D'OR :

- Pour traiter : toujours travailler en WAV (PCM non compressé)
- Pour stocker : FLAC (sans perte) ou MP3/AAC à 192–320 kbps
- Ne jamais re-compresser un MP3 en MP3 : dégradation cumulative

INSTALLATION :

```

pip install pyaudio      # Enregistrement et lecture temps réel
pip install pydub        # Manipulation, découpe, mixage
pip install numpy scipy  # Analyse et traitement du signal
pip install matplotlib   # Visualisation waveform / spectrogramme
sudo apt install ffmpeg  # FFmpeg (requis par pydub pour certains formats)
sudo apt install portaudio19-dev # Requis par pyaudio sous Linux

```

PARTIE C-II – PYAUDIO : ENREGISTREMENT ET LECTURE EN TEMPS RÉEL

C-II.1 ENREGISTRER DEPUIS LE MICROPHONE

```

import pyaudio
import wave

# Paramètres
FORMAT = pyaudio.paInt16 # 16 bits par sample
CHANNELS = 1             # Mono
RATE = 44100             # 44.1 kHz (CD quality)
CHUNK = 1024             # Taille du buffer en samples
DUREE = 10               # Secondes d'enregistrement
FICHIER = "enregistrement.wav"

p = pyaudio.PyAudio()

# Ouvrir le flux d'entrée (microphone)
stream = p.open(
    format=FORMAT,
    channels=CHANNELS,
    rate=RATE,
    input=True,
    frames_per_buffer=CHUNK
)

print(f"Enregistrement en cours ({DUREE}s)...")
frames = []

for _ in range(int(RATE / CHUNK * DUREE)):
    data = stream.read(CHUNK)
    frames.append(data)

```

```

print("Enregistrement terminé.")

stream.stop_stream()
stream.close()
p.terminate()

# Sauvegarder en WAV
with wave.open(FICHER, "wb") as wf:
    wf.setnchannels(CHANNELS)
    wf.setsampwidth(p.get_sample_size(FORMAT))
    wf.setframerate(RATE)
    wf.writeframes(b"".join(frames))

print(f"Sauvegardé : {FICHER}")

```

C-II.2 LIRE UN FICHER AUDIO AVEC PYAUDIO

```

import pyaudio
import wave

FICHER = "audio.wav"
CHUNK = 1024

with wave.open(FICHER, "rb") as wf:
    p = pyaudio.PyAudio()

    stream = p.open(
        format = p.get_format_from_width(wf.getsampwidth()),
        channels = wf.getnchannels(),
        rate = wf.getframerate(),
        output = True
    )

    print(f"Lecture : {FICHER}")
    data = wf.readframes(CHUNK)

    while data:
        stream.write(data)
        data = wf.readframes(CHUNK)

    stream.stop_stream()
    stream.close()
    p.terminate()

print("Lecture terminée.")

```

C-II.3 LISTER LES PÉRIPHÉRIQUES AUDIO

```

import pyaudio

p = pyaudio.PyAudio()

print(f"Nombre de périphériques : {p.get_device_count()}")
print()

for i in range(p.get_device_count()):
    info = p.get_device_info_by_index(i)
    print(f"[{i}] {info['name']}")
    print(f"    Entrées max : {info['maxInputChannels']}")
    print(f"    Sorties max : {info['maxOutputChannels']}")
    print(f"    Sample rate : {info['defaultSampleRate']}")
    print()

# Utiliser un périphérique spécifique
DEVICE_INDEX = 2 # Index du micro souhaité
stream = p.open(
    format=pyaudio.paInt16,
    channels=1,
    rate=44100,
    input=True,
    input_device_index=DEVICE_INDEX,

```

```

frames_per_buffer=1024
)

p.terminate()

```

PARTIE C-III – PYDUB : MANIPULATION, DÉCOUPE, MIXAGE ET EFFETS

C-III.1 BASES DE PYDUB

PRÉREQUIS :
 Pydub utilise FFmpeg pour les formats non-WAV (MP3, AAC, OGG...).
 FFmpeg doit être installé : `sudo apt install ffmpeg`

```

from pydub import AudioSegment

# Charger un fichier audio
audio_mp3 = AudioSegment.from_mp3("musique.mp3")
audio_wav = AudioSegment.from_wav("enregistrement.wav")
audio_aac = AudioSegment.from_file("podcast.aac", format="aac")
audio_ogg = AudioSegment.from_ogg("son.ogg")

# Propriétés
print(audio_mp3.duration_seconds) # Durée en secondes
print(audio_mp3.frame_rate)      # Sample rate (Hz)
print(audio_mp3.channels)        # 1 = mono, 2 = stéréo
print(audio_mp3.sample_width)    # En octets (2 = 16 bits)
print(len(audio_mp3))            # Durée en millisecondes

# Exporter
audio_mp3.export("sortie.wav", format="wav")
audio_mp3.export("sortie.mp3", format="mp3", bitrate="192k")
audio_mp3.export("sortie.ogg", format="ogg")
audio_mp3.export("sortie.flac", format="flac")

```

C-III.2 DÉCOUPE ET ASSEMBLAGE

```

from pydub import AudioSegment

audio = AudioSegment.from_mp3("podcast.mp3")

# — Découpe (en millisecondes) —————
debut    = audio[:30000]          # 0 à 30 secondes
milieu   = audio[30000:90000]     # 30s à 90s
fin      = audio[-60000:]        # 60 dernières secondes
extrait  = audio[60000:120000]    # De 1min à 2min

# — Assemblage —————
montage = debut + milieu + fin
montage.export("montage.mp3", format="mp3")

# — Répéter un segment —————
refrain_x3 = extrait * 3
refrain_x3.export("refrain_repete.mp3", format="mp3")

# — Silence —————
from pydub.generators import Sine

silence_1s = AudioSegment.silent(duration=1000) # 1 seconde de silence
avec_pause = debut + silence_1s + fin

# — Concatener une liste —————
segments = [AudioSegment.from_mp3(f"segment_{i}.mp3") for i in range(5)]
silence   = AudioSegment.silent(duration=500) # 0.5s entre chaque
assemblage = silence.join(segments)
assemblage.export("emission.mp3", format="mp3")

```

C-III.3 VOLUME, EFFETS ET MIXAGE

```

from pydub import AudioSegment
from pydub.effects import normalize, compress_dynamic_range

audio = AudioSegment.from_mp3("audio.mp3")

# — Volume —————
plus_fort = audio + 6      # +6 dB
moins_fort = audio - 10    # -10 dB

# — Normalisation (ramène au volume maximal sans saturation) —————
audio_norm = normalize(audio)

# — Fade in / Fade out —————
audio_fadein = audio.fade_in(2000)      # 2 secondes de fondu entrant
audio_fadeout = audio.fade_out(3000)     # 3 secondes de fondu sortant
audio_fades = audio.fade_in(2000).fade_out(3000)

# — Inverser l'audio —————
audio_inverse = audio.reverse()

# — Compression dynamique —————
audio_comprime = compress_dynamic_range(audio, threshold=-20.0, ratio=4.0)

# — Mixage (superposition) —————
voix = AudioSegment.from_wav("voix.wav")
musique = AudioSegment.from_mp3("fond.mp3") - 15 # Baisser la musique de 15dB

# Ajuster les durées
if len(musique) < len(voix):
    musique = musique * (len(voix) // len(musique) + 1)
musique = musique[:len(voix)]

mixage = voix.overlay(musique)
mixage.export("podcast_avec_musique.mp3", format="mp3", bitrate="192k")

# — Panoramique stéréo (pan) —————
# -1.0 = tout à gauche, 0.0 = centré, 1.0 = tout à droite
audio_gauche = audio.pan(-0.5)
audio_droite = audio.pan(0.5)

# — Changement de sample rate —————
audio_48k = audio.set_frame_rate(48000)
audio_22k = audio.set_frame_rate(22050)

# — Conversion mono ↔ stéréo —————
audio_mono = audio.set_channels(1)
audio_stereo = audio.set_channels(2)

```

C-III.4 DÉTECTION ET DÉCOUPE PAR SILENCE

```

from pydub import AudioSegment
from pydub.silence import split_on_silence, detect_silence

audio = AudioSegment.from_mp3("interview.mp3")

# — Découper sur les silences (utile pour segmenter des interviews) —————
# min_silence_len : durée minimale du silence en ms pour couper
# silence_thresh : seuil en dBFS en dessous duquel c'est "silence"
# keep_silence : garder N ms de silence au début/fin de chaque segment

segments = split_on_silence(
    audio,
    min_silence_len=500,
    silence_thresh=-40,
    keep_silence=200
)

print(f"{len(segments)} segment(s) détecté(s)")

for i, seg in enumerate(segments):
    seg.export(f"segment_{i:03d}.mp3", format="mp3")

```



```
# — Détecter les positions des silences _____
silences = detect_silence(audio, min_silence_len=500, silence_thresh=-40)
for start, end in silences:
    print(f"Silence : {start/1000:.2f}s → {end/1000:.2f}s  ({(end-start)/1000:.2f}s)")

# — Supprimer les silences (montage serré) _____
sans_silences = AudioSegment.empty()
for seg in segments:
    sans_silences += seg
sans_silences.export("interview_compact.mp3", format="mp3")
```

PARTIE C-IV – FFMPEG AUDIO : CONVERSION, EXTRACTION, TRAITEMENT

C-IV.1 OPÉRATIONS FONDAMENTALES SUR L'AUDIO

```
# Informations sur un fichier audio
ffprobe -v quiet -print_format json -show_streams audio.mp3

# Convertir un format
ffmpeg -i audio.mp3 sortie.wav
ffmpeg -i audio.wav -c:a libmp3lame -b:a 192k sortie.mp3
ffmpeg -i audio.wav -c:a aac -b:a 192k sortie.aac
ffmpeg -i audio.mp3 -c:a flac sortie.flac

# Découper
ffmpeg -ss 00:01:00 -to 00:03:30 -i audio.mp3 -c copy extrait.mp3

# Changer le volume
ffmpeg -i audio.mp3 -af "volume=2.0" plus_fort.mp3      # x2
ffmpeg -i audio.mp3 -af "volume=0.5" moins_fort.mp3    # x0.5
ffmpeg -i audio.mp3 -af "volume=6dB" plus_6db.mp3      # +6 dB

# Normaliser le volume (loudnorm)
ffmpeg -i audio.mp3 -af loudnorm=I=-16:TP=-1.5:LRA=11 normalise.mp3

# Changer la vitesse sans changer la hauteur
ffmpeg -i audio.mp3 -af "atempo=1.5" rapide.mp3        # 1.5× plus vite
ffmpeg -i audio.mp3 -af "atempo=0.8" lent.mp3          # 0.8× (ralenti)
# Pour des facteurs > 2 ou < 0.5, chaîner :
ffmpeg -i audio.mp3 -af "atempo=2.0,atempo=2.0" 4x.mp3

# Inverser l'audio
ffmpeg -i audio.mp3 -af areverse inverse.mp3

# Conversion mono
ffmpeg -i stereo.mp3 -ac 1 mono.mp3

# Changer le sample rate
ffmpeg -i audio.mp3 -ar 44100 audio_44k.mp3
ffmpeg -i audio.mp3 -ar 16000 audio_16k.wav            # Pour reconnaissance vocale
```

C-IV.2 FILTRES AUDIO AVANCÉS

```
# Réduction de bruit (high-pass : supprimer les basses fréquences parasites)
ffmpeg -i audio.mp3 -af "highpass=f=200" sans_bruit_bas.mp3

# Égalisation (EQ) – atténuer ou amplifier des fréquences
ffmpeg -i audio.mp3 -af "equalizer=f=1000:width_type=o:width=2:g=5" eq.mp3
# f=1000 : fréquence centrale 1000Hz, g=5 : gain +5dB

# Suppression du bruit de fond (nlfilt, ou avec anlmdn)
ffmpeg -i bruyant.wav -af anlmdn=s=7:p=0.002:r=0.002:l=5 propre.wav

# Compresseur dynamique
ffmpeg -i audio.mp3 -af "acompressor=threshold=0.5:ratio=4:attack=5:release=50" comp.mp3

# Reverb (réverbération)
ffmpeg -i audio.wav -af "aecho=0.8:0.88:60:0.4" reverb.wav
```

```
# Fade in / Fade out
ffmpeg -i audio.mp3 -af "afade=t=in:ss=0:d=3" fadein.mp3      # 3s de fade in
ffmpeg -i audio.mp3 -af "afade=t=out:st=27:d=3" fadeout.mp3   # Fade out à partir de 27s

# Mixage de deux pistes
ffmpeg -i voix.mp3 -i musique.mp3 \
  -filter_complex "[0:a]volume=1.0[a1];[1:a]volume=0.2[a2];[a1][a2]amix=inputs=2[out]" \
  -map "[out]" -c:a libmp3lame -b:a 192k mixage.mp3

# Concaténer des fichiers audio
# Créer liste.txt
echo "file 'partiel.mp3'" > liste.txt
echo "file 'partie2.mp3'" >> liste.txt
echo "file 'partie3.mp3'" >> liste.txt
ffmpeg -f concat -safe 0 -i liste.txt -c copy emission_complete.mp3
```

C-IV.3 FFMPEG AUDIO DEPUIS PYTHON

```
import subprocess
import os

def ffmpeg_audio(commande):
    result = subprocess.run(commande, capture_output=True, text=True)
    if result.returncode != 0:
        print("Erreur FFmpeg :", result.stderr[-300:])
    return result.returncode == 0

# Convertir tous les WAV d'un dossier en MP3
dossier = "enregistrements/"
sortie = "mp3_exports/"
os.makedirs(sortie, exist_ok=True)

for f in os.listdir(dossier):
    if f.endswith(".wav"):
        entree = os.path.join(dossier, f)
        out_f = os.path.join(sortie, f.replace(".wav", ".mp3"))
        ok = ffmpeg_audio([
            "ffmpeg", "-y", "-i", entree,
            "-c:a", "libmp3lame", "-b:a", "192k", out_f
        ])
        print(f"{'✓' if ok else '✗'} {f} → {out_f}")
```

PARTIE C-V – ANALYSE AUDIO : WAVEFORM, SPECTROGRAMME, FRÉQUENCES

C-V.1 VISUALISER LA WAVEFORM (FORME D'ONDE)

```
import numpy as np
import matplotlib.pyplot as plt
import wave

def afficher_waveform(chemin_wav, titre="Waveform"):
    with wave.open(chemin_wav, "rb") as wf:
        n_frames = wf.getnframes()
        sample_rate = wf.getframerate()
        n_canaux = wf.getnchannels()
        raw_data = wf.readframes(n_frames)

    # Convertir en tableau NumPy
    signal = np.frombuffer(raw_data, dtype=np.int16)

    # Stéréo → mono (moyenne des deux canaux)
    if n_canaux == 2:
        signal = signal.reshape(-1, 2).mean(axis=1)

    # Axe temporel
    duree = len(signal) / sample_rate
    temps = np.linspace(0, duree, len(signal))

    # Tracé
```

```

plt.figure(figsize=(14, 4))
plt.plot(temps, signal, color="#1A8FA0", linewidth=0.5)
plt.title(titre, fontsize=14, fontweight="bold")
plt.xlabel("Temps (secondes)")
plt.ylabel("Amplitude")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("waveform.png", dpi=150)
plt.show()
print(f"Durée : {duree:.2f}s | Sample rate : {sample_rate} Hz | Canaux : {n_canaux}")

afficher_waveform("enregistrement.wav", titre="Enregistrement GCI")

```

C-V.2 SPECTROGRAMME (ANALYSE FRÉQUENTIELLE DANS LE TEMPS)

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import spectrogram

def afficher_spectrogramme(chemin_wav):
    sample_rate, signal = wavfile.read(chemin_wav)

    # Mono si stéréo
    if signal.ndim == 2:
        signal = signal.mean(axis=1)

    signal = signal.astype(np.float32)

    # Calcul du spectrogramme
    freqs, temps, Sxx = spectrogram(signal, fs=sample_rate, nperseg=1024)

    # Convertir en dB
    Sxx_db = 10 * np.log10(Sxx + 1e-10)

    plt.figure(figsize=(14, 6))
    plt.pcolormesh(temps, freqs, Sxx_db, shading="gouraud", cmap="inferno")
    plt.colorbar(label="Intensité (dB)")
    plt.ylabel("Fréquence (Hz)")
    plt.xlabel("Temps (s)")
    plt.title("Spectrogramme", fontsize=14, fontweight="bold")
    plt.ylim(0, 8000) # Limiter à 8 kHz pour la voix
    plt.tight_layout()
    plt.savefig("spectrogramme.png", dpi=150)
    plt.show()

afficher_spectrogramme("enregistrement.wav")

```

C-V.3 ANALYSE FFT : FRÉQUENCES DOMINANTES

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

def analyser_frequences(chemin_wav, n_top=10):
    sample_rate, signal = wavfile.read(chemin_wav)

    if signal.ndim == 2:
        signal = signal.mean(axis=1)

    signal = signal.astype(np.float32)

    # FFT (Fast Fourier Transform)
    N = len(signal)
    fft = np.fft.rfft(signal)
    freqs = np.fft.rfftfreq(N, d=1.0 / sample_rate)
    ampls = np.abs(fft) / N

    # Top fréquences
    indices_top = np.argsort(ampls)[::-1][:n_top]
    print(f"\nTop {n_top} fréquences dominantes :")

```

```

for i, idx in enumerate(indices_top):
    print(f" {i+1:2d}. {freqs[idx]:8.1f} Hz - Amplitude : {ampls[idx]:.2f}")

# Tracé du spectre
plt.figure(figsize=(14, 5))
plt.plot(freqs, ampls, color="#B8912A", linewidth=0.8)
plt.title("Spectre de fréquences (FFT)", fontsize=14, fontweight="bold")
plt.xlabel("Fréquence (Hz)")
plt.ylabel("Amplitude")
plt.xlim(0, 8000)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("spectre_fft.png", dpi=150)
plt.show()

analyser_frequences("enregistrement.wav")

```

PARTIE C-VI – PROJETS MÉTIER AUDIO

C-VI.1 TRANSCRIPTION AUTOMATIQUE AVEC WHISPER (OPENAI)

CONCEPT :

Whisper est le modèle de transcription audio d'OpenAI. Il tourne entièrement en local, supporte le français, et est gratuit.

```

pip install openai-whisper
sudo apt install ffmpeg # Requis

```

```

import whisper

```

```

# Charger un modèle (tiny, base, small, medium, large)
# Plus le modèle est grand, plus c'est précis mais lent
model = whisper.load_model("small") # Bon compromis vitesse/précision

```

```

# Transcrire
result = model.transcribe("enregistrement.mp3", language="fr")
print(result["text"])

```

```

# Sauvegarder la transcription
with open("transcription.txt", "w", encoding="utf-8") as f:
    f.write(result["text"])

```

```

# Obtenir les segments avec timestamps
for segment in result["segments"]:
    debut = segment["start"]
    fin = segment["end"]
    texte = segment["text"]
    print(f"[{debut:6.2f}s - {fin:6.2f}s] {texte}")

```

```

# Export SRT (sous-titres)
with open("sous_titres.srt", "w", encoding="utf-8") as f:
    for i, seg in enumerate(result["segments"], 1):
        def fmt(s):
            h = int(s // 3600)
            m = int((s % 3600) // 60)
            sec = s % 60
            return f"{h:02d}:{m:02d}:{sec:06.3f}".replace(".", ",")
        f.write(f"{i}\n{fmt(seg['start'])} --> {fmt(seg['end'])}\n{seg['text'].strip()}\n\n")

print("Transcription et sous-titres générés.")

```

C-VI.2 PIPELINE COMPLET : DÉCOUPE + TRANSCRIPTION + EXPORT

SCÉNARIO MÉTIER :

Vous recevez l'enregistrement d'une réunion client (1h).
Objectif : extraire les segments parlés, transcrire, produire un rapport.

```

import subprocess, os, json
from pydub import AudioSegment

```

```

from pydub.silence import split_on_silence

def pipeline_reunion(fichier_audio, dossier_sortie="rapport_reunion/"):

    os.makedirs(dossier_sortie, exist_ok=True)

    # 1. Charger et convertir en WAV si nécessaire
    print("1. Chargement de l'audio...")
    audio = AudioSegment.from_file(fichier_audio)
    wav_path = os.path.join(dossier_sortie, "reunion.wav")
    audio.export(wav_path, format="wav")

    # 2. Normaliser le volume
    print("2. Normalisation...")
    norm_path = os.path.join(dossier_sortie, "reunion_norm.wav")
    subprocess.run([
        "ffmpeg", "-y", "-i", wav_path,
        "-af", "loudnorm=I=-16:TP=-1.5:LRA=11", norm_path
    ], capture_output=True)

    # 3. Découper sur les silences
    print("3. Découpe des segments...")
    audio_norm = AudioSegment.from_wav(norm_path)
    segments = split_on_silence(
        audio_norm, min_silence_len=800, silence_thresh=-38, keep_silence=300
    )
    print(f" → {len(segments)} segment(s) détecté(s)")

    segments_paths = []
    for i, seg in enumerate(segments):
        if len(seg) < 2000: # Ignorer les segments < 2s
            continue
        chemin = os.path.join(dossier_sortie, f"seg_{i:04d}.wav")
        seg.export(chemin, format="wav")
        segments_paths.append(chemin)

    # 4. Transcrire avec Whisper
    print("4. Transcription...")
    try:
        import whisper
        model = whisper.load_model("small")
        transcriptions = []
        for chemin in segments_paths:
            r = model.transcribe(chemin, language="fr")
            transcriptions.append(r["text"].strip())
    except ImportError:
        transcriptions = ["[Whisper non installé]" for _ in segments_paths]

    # 5. Générer le rapport texte
    print("5. Génération du rapport...")
    rapport_path = os.path.join(dossier_sortie, "rapport_transcription.txt")
    with open(rapport_path, "w", encoding="utf-8") as f:
        f.write("=" * 70 + "\n")
        f.write("GCI – RAPPORT DE TRANSCRIPTION RÉUNION\n")
        f.write("=" * 70 + "\n\n")
        f.write(f"Fichier source      : {fichier_audio}\n")
        f.write(f"Durée totale       : {audio.duration_seconds:.1f}s\n")
        f.write(f"Segments parlés    : {len(transcriptions)}\n")
        f.write("-" * 70 + "\n\n")
        for i, texte in enumerate(transcriptions):
            f.write(f"[SEGMENT {i+1:03d}]\n{texte}\n\n")
        f.write("=" * 70 + "\n")
        f.write("FIN DU RAPPORT – GCI\n")
        f.write("=" * 70 + "\n")

    print(f"\n✓ Rapport généré : {rapport_path}")
    print(f"✓ {len(transcriptions)} segments transcrits")

    pipeline_reunion("reunion_client.mp3")

```

C-VI.3 ERREURS FRÉQUENTES ET BONNES PRATIQUES AUDIO

ERREUR 1 – PyAudio introuvable sous Linux

→ `sudo apt install portaudio19-dev && pip install pyaudio`

ERREUR 2 – Pydub ne peut pas ouvrir un MP3

→ FFmpeg n'est pas installé ou pas dans le PATH.

→ `sudo apt install ffmpeg`

ERREUR 3 – Re-compression MP3 → MP3

→ Toujours décompresser en WAV avant de traiter, puis recompresser en MP3 seulement pour l'export final.

ERREUR 4 – Fichier WAV tronqué ou corrompu

→ Toujours fermer les streams PyAudio avec `stream.stop_stream()` et `stream.close()` avant `p.terminate()`.

ERREUR 5 – Whisper très lent

→ Utiliser le modèle "tiny" ou "base" pour des tests rapides.

→ Passer à "small" ou "medium" pour la production.

→ Installer whisper avec support GPU si disponible.

ERREUR 6 – Silence non détecté correctement

→ Ajuster `silence_thresh` selon l'environnement. En studio : -50 dBFS.

En bureau : -35 à -40 dBFS. Vérifier avec `detect_silence()` d'abord.

BONNES PRATIQUES :

– Toujours travailler en WAV (PCM) pour les traitements intermédiaires

– Normaliser avant toute analyse (`loudnorm FFmpeg`)

– Conserver l'original intact, travailler sur des copies

– Documenter le sample rate dans les noms de fichiers (ex: `audio_44k.wav`)

– Tester les découpes sur de courts extraits avant de lancer en batch

RESSOURCES COMPLÉMENTAIRES

IMAGE / INFOGRAPHIE :

– Documentation OpenCV : docs.opencv.org

– Documentation Pillow : pillow.readthedocs.io

– OpenCV Python Tutorials : docs.opencv.org/4.x/d9/df8/tutorial_root.html

– "Computer Vision with Python and OpenCV" – Packt Publishing

VIDÉO :

– Documentation MoviePy : zulko.github.io/moviepy

– Documentation FFmpeg : ffmpeg.org/documentation.html

– FFmpeg Filters Guide : ffmpeg.org/ffmpeg-filters.html

AUDIO :

– Documentation PyAudio : people.csail.mit.edu/hubert/pyaudio/docs

– Documentation Pydub : github.com/jiaaro/pydub

– OpenAI Whisper : github.com/openai/whisper

– `scipy.signal` : docs.scipy.org/doc/scipy/reference/signal.html

– Cours DSP (Digital Signal Processing) : dspguide.com

FIN DU COURS – GCI © 2024-2025 – TOUS DROITS RÉSERVÉS

Formation continue disponible via GCI – Congo-Brazzaville
